

user

java多线程与并发编程 Thread、Runnable、Callable 线程池（ExecutorService、ThreadPoolExecutor）
并发工具类（CountDownLatch、CyclicBarrier、Semaphore、ReentrantLock、ReadWriteLock）
volatile、synchronized、CAS 原理 JUC（java.util.concurrent）包常用类 死锁、活锁、线程安全问题排查

详细介绍这些，并且从面试的角度来举例问题和解答问题，以及解答问题的思路

assistant

Java 多线程与并发编程是 Java 高级开发工程师面试中的核心内容，不仅考察对语言特性的掌握，更关注对底层原理、并发模型和实际问题排查的理解。以下从概念讲解 + 面试高频问题 + 解答思路 + 示例代码的角度系统梳理。

一、Thread、Runnable、Callable

1. 基本区别

类型	特点	返回值	异常处理
Thread	继承 Thread 类（不推荐）	无	不能抛受检异常
<	Runnable	无	实现接口（推荐）
Callable	实现接口，配合 FutureTask 使用	有（通过 Future.get() 获取）	可抛受检异常

2. 面试问题示例

Q：为什么推荐使用 Runnable 而不是继承 Thread？

✓ 答：

- Java 不支持多继承，继承 Thread 会占用唯一继承机会；
- Runnable 更符合“任务”与“执行”的解耦思想；
- 可复用性更强，同一个 Runnable 可被多个线程执行或提交到线程池。

Q：Callable 和 Runnable 的主要区别是什么？如何获取 Callable 的返回值？

✓ 答：

- Callable 的 call() 方法可以返回结果并抛出异常；
- 通常通过 FutureTask 包装 Callable，再交给 Thread 执行，或直接提交到线程池（如 ExecutorService.submit(callable)），返回 Future 对象；

- 调用 `future.get()` 阻塞等待结果（可设超时）。

```
ExecutorService executor = Executors.newSingleThreadExecutor();
Future<Integer> future = executor.submit(() -> {
    return 42;
});
System.out.println(future.get()); // 输出 42
```

二、线程池（ExecutorService、ThreadPoolExecutor）

1. 核心类关系

- `Executor`：最顶层接口，定义 `execute(Runnable)`
- `ExecutorService`：扩展了 `submit`、`shutdown` 等方法
- `ThreadPoolExecutor`：核心实现类，可定制线程池行为

2. ThreadPoolExecutor 构造参数

```
new ThreadPoolExecutor(
    int corePoolSize, // 核心线程数
    int maximumPoolSize, // 最大线程数
    long keepAliveTime, // 非核心线程空闲存活时间
    TimeUnit unit,
    BlockingQueue<Runnable> workQueue, // 任务队列
    ThreadFactory threadFactory,
    RejectedExecutionHandler handler // 拒绝策略
);
```

3. 常见拒绝策略

- `AbortPolicy`（默认）：抛 `RejectedExecutionException`
- `CallerRunsPolicy`：由调用线程执行任务
- `DiscardPolicy`：静默丢弃
- `DiscardOldestPolicy`：丢弃队列中最老的任务

4. 面试问题示例

Q：线程池的 `execute()` 和 `submit()` 有什么区别？

✅ 答：

- `execute()` 只接受 `Runnable`，无返回值；
- `submit()` 可接受 `Runnable/Callable`，返回 `Future`，可用于获取结果或取消任务。

Q：FixedThreadPool 会导致 OOM 吗？为什么？



答：

- 会！ Executors.newFixedThreadPool(n) 使用的是 无界 LinkedBlockingQueue；
- 如果任务提交速度远大于处理速度，队列无限增长，最终导致内存溢出；
- 建议：生产环境应使用 ThreadPoolExecutor 显式指定有界队列和拒绝策略。

三、并发工具类（JUC 工具）

1. CountdownLatch

- 作用：一个或多个线程等待其他线程完成操作。
- 核心方法： countDown(), await()
- 不可重用

```
CountDownLatch latch = new CountDownLatch(3);  
// 3个线程执行完后，主线程继续  
latch.await();
```

2. CyclicBarrier

- 作用：多个线程互相等待，达到屏障点后一起继续。
- 可重用
- 可设置 barrierAction（所有线程到达后执行）

```
CyclicBarrier barrier = new CyclicBarrier(3, () -> System.out.println("All ready!"));  
barrier.await(); // 每个线程调用
```

3. Semaphore

- 作用：控制同时访问特定资源的线程数量（信号量）
- 常用于限流、资源池控制

```
Semaphore sem = new Semaphore(2); // 允许2个并发  
sem.acquire(); // 获取许可  
// ... 访问资源  
sem.release(); // 释放
```

4. ReentrantLock

- 可重入、可中断、可设置公平锁
- 支持 Condition（替代 wait/notify）

5. ReadWriteLock

- 读写分离锁：多个读线程可并发，写线程独占
- 实现类： `ReentrantReadWriteLock`

面试问题示例

Q : `CountDownLatch` 和 `CyclicBarrier` 的区别？

✓ 答：

- `CountDownLatch` 是一次性的，计数减到0后无法重置；
- `CyclicBarrier` 是可循环使用的，适用于多轮同步；
- `CountDownLatch` 通常用于主线程等待子线程，`CyclicBarrier` 用于多个子线程相互等待。

Q : `Semaphore` 能用来实现互斥锁吗？

✓ 答：

- 可以！将信号量初始化为 1 (`new Semaphore(1)`)，就等价于互斥锁；
- 但 `ReentrantLock` 功能更强大（可重入、条件变量等），一般优先用 `Lock`。

四、volatile、synchronized、CAS 原理

1. volatile

- 作用：保证可见性和禁止指令重排序，但不保证原子性
- 底层：通过内存屏障（`Memory Barrier`）实现
- 适用场景：状态标志、单例 double-check 中的 instance 引用

2. synchronized

- 作用：保证原子性、可见性、有序性
- JVM 层面通过 `Monitor`（对象头 `Mark Word` + `Monitor Record`）实现
- JDK6 后优化：偏向锁 → 轻量级锁 → 重量级锁

3. CAS（Compare And Swap）

- 原理：CPU 原子指令（如 `cmpxchg`），实现无锁并发
- ABA 问题：可用 `AtomicStampedReference` 解决
- JUC 中大量使用：`AtomicInteger`、`ConcurrentHashMap` 等

面试问题示例

Q : `volatile` 能保证 `i++` 的线程安全吗？为什么？

✓ 答：

- 不能！`i++` 是读-改-写三步操作，`volatile` 只保证每次读取的是最新值，但中间可能被其他线程修改；

- 需要使用 `synchronized` 或 `AtomicInteger` 。

Q : `synchronized` 和 `ReentrantLock` 的区别？

✔ 答：

特性	synchronized	ReentrantLock
实现	JVM 内置	API 层
锁释放	自动	必须手动（try-finally）
可中断	否	是（lockInterruptibly）
超时	否	是（tryLock(timeout)）
公平性	否	可选
条件变量	1 个（wait/notify）	多个（Condition）

五、JUC 常用类

类	用途
ConcurrentHashMap	高并发 HashMap，分段锁（JDK7）→ CAS + synchronized（JDK8）
CopyOnWriteArrayList	读多写少场景，写时复制
BlockingQueue	生产者-消费者模型（ArrayBlockingQueue, LinkedBlockingQueue）
CompletableFuture	异步编程，链式调用（Java8+）
AtomicInteger / AtomicReference	原子操作，基于 CAS

面试题：

Q：ConcurrentHashMap 为什么比 Hashtable 高效？

✔ 答：

- Hashtable 对整个 map 加锁，ConcurrentHashMap 只锁部分（JDK7 分段，JDK8 锁桶头节点）；
- 支持更高并发度。

六、死锁、活锁、线程安全问题排查

1. 死锁（Deadlock）

- 四个必要条件：互斥、持有并等待、不可剥夺、循环等待
- 避免方法：按固定顺序加锁、使用超时、避免嵌套锁

死锁检测：

```
jstack `cat pid` # 查看线程堆栈，找 "Found one Java-level deadlock"
```

2. 活锁 (Livelock)

- 线程不断改变状态以响应对方，但无法前进（如两人在走廊互相避让）
- 比死锁更难发现，可通过引入随机退避解决

3. 线程安全问题排查思路

1. 现象：数据错乱、程序卡死、CPU 100%

2. 工具：

- jstack：看线程状态（BLOCKED、WAITING）
- jconsole / VisualVM：监控线程、内存
- Arthas：线上诊断（thread -b 查死锁）

3. 代码审查：

- 是否共享可变状态？
- 是否正确加锁？
- 是否使用了非线程安全类（如 SimpleDateFormat）？

面试问题示例

Q：如何编写一个死锁程序？

✅ 答：

```
Object lock1 = new Object();
Object lock2 = new Object();

new Thread(() -> {
    synchronized (lock1) {
        Thread.sleep(100);
        synchronized (lock2) {}
    }
}).start();

new Thread(() -> {
    synchronized (lock2) {
        Thread.sleep(100);
        synchronized (lock1) {}
    }
}).start();
```

Q：如何避免 SimpleDateFormat 的线程安全问题？

✓ 答：

- 使用 ThreadLocal 封装；
- 使用 Java8 的 DateTimeFormatter （线程安全）；
- 每次新建实例（性能差）。

总结：面试答题思路

1. 先说本质：这个机制/类是用来解决什么问题的？
2. 对比差异：和类似方案比，优势在哪？（如 Callable vs Runnable）
3. 讲清原理：底层怎么实现？（如 volatile 内存屏障，synchronized Monitor）
4. 举反例/陷阱：常见误用（如 volatile 用于 i++）
5. 结合实践：如何排查？如何优化？（如线程池参数设置、死锁检测）

如果你正在准备面试，建议：

- 手写线程池创建、双检锁单例（带 volatile）
- 能画出 JUC 类图关系
- 熟悉 jstack、jconsole 使用

需要我提供模拟面试题集或代码模板，也可以继续提问！

user

手写线程池创建、双检锁单例（带 volatile）能画出 JUC 类图关系 熟悉 jstack、jconsole 使用

assistant

手写线程池创建

Java中可以通过 ThreadPoolExecutor 类自定义线程池。以下是一个简单的例子：

```
import java.util.concurrent.*;

public class CustomThreadPoolExample {
    public static void main(String[] args) {
        // 创建线程池
        ThreadPoolExecutor executor = new ThreadPoolExecutor(
            2, // 核心线程数
            4, // 最大线程数
            1L, TimeUnit.SECONDS, // 非核心线程空闲时间
            new LinkedBlockingQueue<>(10), // 任务队列
            new ThreadPoolExecutor.CallerRunsPolicy() // 拒绝策略
        );
    }
}
```

```

// 提交任务
for (int i = 0; i < 10; i++) {
    final int taskNumber = i;
    executor.submit(() -> {
        System.out.println("Executing Task " + taskNumber);
        try {
            Thread.sleep(2000); // 模拟任务执行
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    });
}

// 关闭线程池
executor.shutdown();
}
}

```

双检锁单例（带 volatile）

在Java中，使用双重检查锁定模式可以确保懒加载的单例对象同时是线程安全的。

```

public class Singleton {
    private static volatile Singleton instance;

    private Singleton() {}

    public static Singleton getInstance() {
        if (instance == null) { // 第一次检查
            synchronized (Singleton.class) {
                if (instance == null) { // 第二次检查
                    instance = new Singleton(); // 使用volatile关键字防止指令重排序
                }
            }
        }
        return instance;
    }
}

```

JUC 类图关系概述

由于文本环境限制，无法直接绘制出详细的JUC类图关系，但这里提供一个简化的结构说明：

- `Executor` 是所有执行器框架的顶层接口。
- `ExecutorService` 继承了 `Executor` 接口，并添加了管理和控制服务的方法。
- `AbstractExecutorService` 提供了 `ExecutorService` 接口的默认实现。
- `ThreadPoolExecutor` 和 `ScheduledThreadPoolExecutor` 是两个常用的执行器实现类，分别用于普通线程池和调度线程池。
- `Executors` 工具类提供了各种静态方法来创建不同类型的线程池。

- 其他并发工具如 `CountDownLatch` , `CyclicBarrier` , `Semaphore` , `ReentrantLock` , `ReadWriteLock` 等属于 `java.util.concurrent` 包下，它们各自解决了不同的同步问题。

jstack、jconsole 使用简介

jstack

`jstack` 是一个命令行工具，用于打印指定 Java 进程中的所有线程堆栈信息。这对于分析死锁或性能瓶颈非常有用。

- 基本用法： `jstack` ，其中 `` 是目标Java进程的ID。
- 查找死锁： `jstack -l` `` 可以显示更详细的信息，包括锁的状态。

jconsole

`jconsole` 是一个图形界面工具，允许用户监控Java应用程序的性能和内存使用情况。它也支持查看线程状态、堆栈跟踪以及垃圾回收活动等。

- 启动方式：从命令行运行 `jconsole` 。
- 连接到应用：启动后选择要连接的Java进程，然后即可查看其概览、线程、类、VM摘要、MBeans等信息。

以上就是关于手写线程池创建、双检锁单例（带 `volatile` ）、JUC 类图关系描述以及 `jstack`、`jconsole` 使用的基本介绍。如果需要进一步的具体操作指导或代码示例，请告知！